

Name: Bestha Akshaya

Course code: CSA0611

Regno:192424336

Course name: DAA

Slot C

LAB EXPERIMENTS

Day -3

1. 1. Write a program to perform the following

- o An empty list**
- o A list with one element**
- o A list with all identical elements**
- o A list with negative numbers**

Test Cases:

1. Input: []

o Expected Output: []

2. Input: [1]

o Expected Output: [1]

3. Input: [7, 7, 7, 7]

o Expected Output: [7, 7, 7, 7]

4. Input: [-5, -1, -3, -2, -4]

o Expected Output: [-5, -4, -3, -2, -1]

Code:

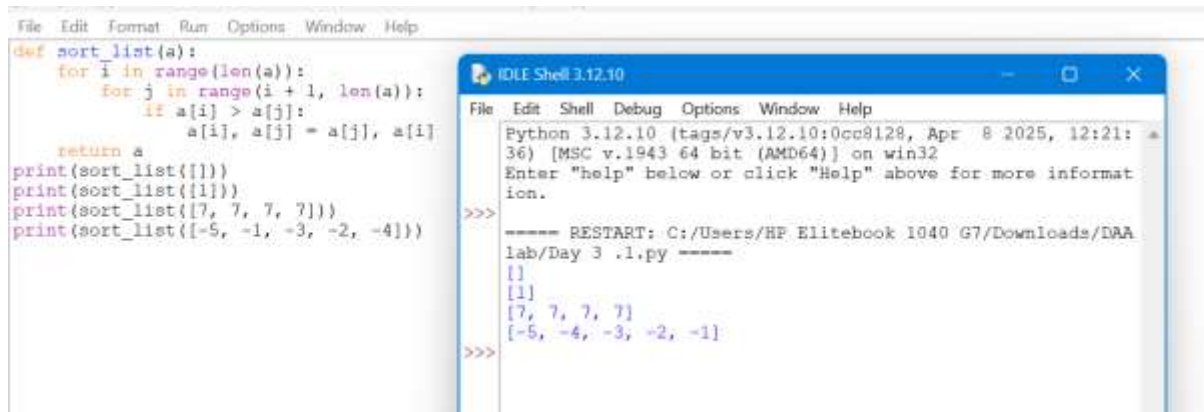
```
def sort_list(a):  
    for i in range(len(a)):  
        for j in range(i + 1, len(a)):  
            if a[i] > a[j]:  
                a[i], a[j] = a[j], a[i]  
    return a
```

```

print(sort_list([]))
print(sort_list([1]))
print(sort_list([7, 7, 7, 7]))
print(sort_list([-5, -1, -3, -2, -4]))

```

output:



The screenshot shows a Python IDE with two windows. The left window displays the implementation of the Selection Sort algorithm, and the right window shows the output of the program.

```

def sort_list(a):
    for i in range(len(a)):
        for j in range(i + 1, len(a)):
            if a[i] > a[j]:
                a[i], a[j] = a[j], a[i]
    return a
print(sort_list([]))
print(sort_list([1]))
print(sort_list([7, 7, 7, 7]))
print(sort_list([-5, -1, -3, -2, -4]))

```

The output window shows the following results:

```

Python 3.12.10 (tags/v3.12.10:0ccc9128, Apr 8 2025, 12:21:
36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more informat
ion.
>>>
----- RESTART: C:/Users/HP Elitebook 1040 G7/Downloads/DAA
lab/Day 3 .1.py -----
[]
[1]
[7, 7, 7, 7]
[-5, -4, -3, -2, -1]
>>>

```

2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

Sorting a Random Array:

Input: [5, 2, 9, 1, 5, 6]

Output: [1, 2, 5, 5, 6, 9]

Sorting a Reverse Sorted Array:

Input: [10, 8, 6, 4, 2]

Output: [2, 4, 6, 8, 10]

Sorting an Already Sorted Array:

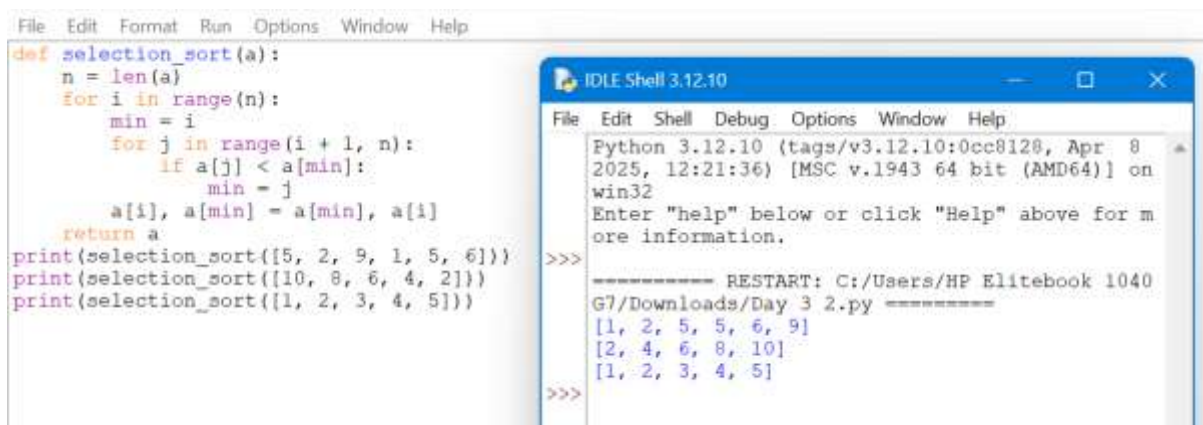
Input: [1, 2, 3, 4, 5]

Output: [1, 2, 3, 4, 5]

Code:

```
def selection_sort(a):  
    n = len(a)  
    for i in range(n):  
        min = i  
        for j in range(i + 1, n):  
            if a[j] < a[min]:  
                min = j  
        a[i], a[min] = a[min], a[i]  
    return a  
  
print(selection_sort([5, 2, 9, 1, 5, 6]))  
print(selection_sort([10, 8, 6, 4, 2]))  
print(selection_sort([1, 2, 3, 4, 5]))
```

Output:

The image shows a screenshot of the Python IDLE environment. On the left, the 'File Editor' window displays the selection_sort function and three print statements. On the right, the 'IDLE Shell 3.12.10' window shows the execution results. The shell output indicates a restart of the file 'C:/Users/HP Elitebook 1040/G7/Downloads/Day 3 2.py' and displays the sorted lists: [1, 2, 5, 5, 6, 9], [2, 4, 6, 8, 10], and [1, 2, 3, 4, 5].

```
File Editor  
def selection_sort(a):  
    n = len(a)  
    for i in range(n):  
        min = i  
        for j in range(i + 1, n):  
            if a[j] < a[min]:  
                min = j  
        a[i], a[min] = a[min], a[i]  
    return a  
print(selection_sort([5, 2, 9, 1, 5, 6]))  
print(selection_sort([10, 8, 6, 4, 2]))  
print(selection_sort([1, 2, 3, 4, 5]))  
  
IDLE Shell 3.12.10  
File Edit Shell Debug Options Window Help  
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
>>>  
----- RESTART: C:/Users/HP Elitebook 1040  
G7/Downloads/Day 3 2.py -----  
[1, 2, 5, 5, 6, 9]  
[2, 4, 6, 8, 10]  
[1, 2, 3, 4, 5]  
>>>
```

3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.

Test Cases:

☐ Test your optimized function with the following lists:

1. Input: [64, 25, 12, 22, 11]

☐ Expected Output: [11, 12, 22, 25, 64]

2. Input: [29, 10, 14, 37, 13]

☐ Expected Output: [10, 13, 14, 29, 37]

3. Input: [3, 5, 2, 1, 4]

☐ **Expected Output: [1, 2, 3, 4, 5]**

4. Input: [1, 2, 3, 4, 5] (Already sorted list)

☐ **Expected Output: [1, 2, 3, 4, 5]**

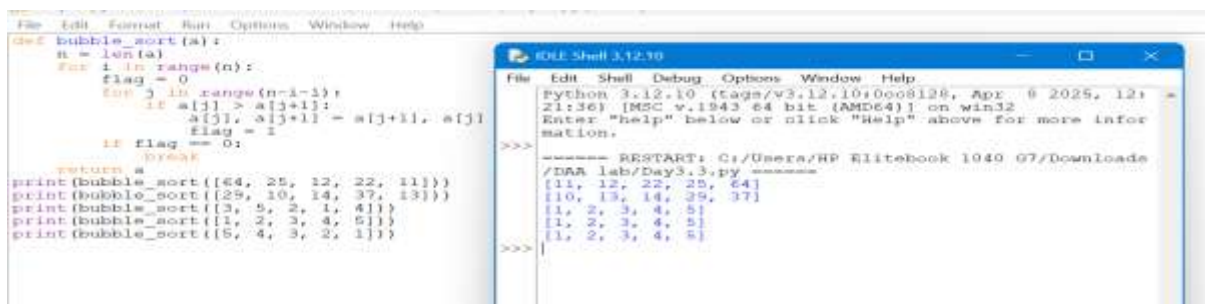
5. Input: [5, 4, 3, 2, 1] (Reverse sorted list)

☐ **Expected Output: [1, 2, 3, 4, 5]**

Code:

```
def bubble_sort(a):  
    n = len(a)  
    for i in range(n):  
        flag = 0  
        for j in range(n-i-1):  
            if a[j] > a[j+1]:  
                a[j], a[j+1] = a[j+1], a[j]  
                flag = 1  
        if flag == 0:  
            break  
    return a  
  
print(bubble_sort([64, 25, 12, 22, 11]))  
print(bubble_sort([29, 10, 14, 37, 13]))  
print(bubble_sort([3, 5, 2, 1, 4]))  
print(bubble_sort([1, 2, 3, 4, 5]))  
print(bubble_sort([5, 4, 3, 2, 1]))
```

output:



The screenshot shows a Python IDE with two windows. The left window displays the bubble_sort function and five print statements. The right window shows the output of these statements, which are the sorted lists: [11, 12, 22, 25, 64], [10, 13, 14, 29, 37], [1, 2, 3, 4, 5], [1, 2, 3, 4, 5], and [1, 2, 3, 4, 5].

```
File Edit Format Run Options Window Help  
def bubble_sort(a):  
    n = len(a)  
    for i in range(n):  
        flag = 0  
        for j in range(n-i-1):  
            if a[j] > a[j+1]:  
                a[j], a[j+1] = a[j+1], a[j]  
                flag = 1  
        if flag == 0:  
            break  
    return a  
  
print(bubble_sort([64, 25, 12, 22, 11]))  
print(bubble_sort([29, 10, 14, 37, 13]))  
print(bubble_sort([3, 5, 2, 1, 4]))  
print(bubble_sort([1, 2, 3, 4, 5]))  
print(bubble_sort([5, 4, 3, 2, 1]))  
  
Python 3.12.10 (tags/v3.12.10:0008126, Apr 6 2025, 12:  
21:36) [MSC v.1943 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more infor  
mation.  
  
>>> ----- RESTART: C:/Users/HP Elitebook 1040 G7/Downloade  
/DAA lab/Day3.3.py -----  
[11, 12, 22, 25, 64]  
[10, 13, 14, 29, 37]  
[1, 2, 3, 4, 5]  
[1, 2, 3, 4, 5]  
[1, 2, 3, 4, 5]  
>>>
```

4. Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome.

Examples:

1. Array with Duplicates:

o Input: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]

o Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

2. All Identical Elements:

o Input: [5, 5, 5, 5, 5]

o Output: [5, 5, 5, 5, 5]

3. Mixed Duplicates:

o Input: [2, 3, 1, 3, 2, 1, 1, 3]

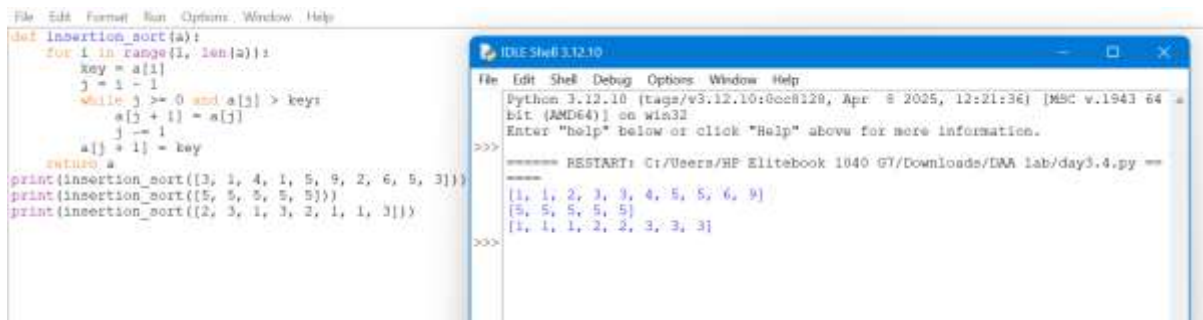
o Output: [1, 1, 1, 2, 2, 3, 3, 3]

Code:

```
def insertion_sort(a):
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a

print(insertion_sort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3]))
print(insertion_sort([5, 5, 5, 5, 5]))
print(insertion_sort([2, 3, 1, 3, 2, 1, 1, 3]))
```

Output:



The screenshot shows an IDE with two windows. The left window displays Python code for an insertion sort function and its application to three arrays. The right window shows the output of the program, which prints the sorted arrays.

```
def insertion_sort(a):
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a

print(insertion_sort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3]))
print(insertion_sort([5, 5, 5, 5, 5]))
print(insertion_sort([2, 3, 1, 3, 2, 1, 1, 3]))
```

```
Python 3.12.10 [tags/v3.12.10:6cc8120, Apr 8 2025, 12:21:36] [MSC v.1943 64-bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/HP Elitebook 1040 G7/Downloads/DAA lab/day3.4.py =====
[1, 1, 2, 3, 9, 4, 5, 5, 6, 9]
[5, 5, 5, 5, 5]
[1, 1, 1, 2, 2, 3, 3, 3]
```

5. Given an array arr of positive integers sorted in a strictly increasing order, and an integer k. return

the kth positive integer that is missing from this array.

Example 1:

Input: arr = [2,3,4,7,11], k = 5

Output: 9

Explanation: The missing positive integers are [1,5,6,8,9,10,12,13,...]. The 5th missing

positive integer is 9.

Example 2:

Input: arr = [1,2,3,4], k = 2

Output: 6

Explanation: The missing positive integers are [5,6,7,...]. The 2nd missing positive

Code:

```
arr = [2, 3, 4, 7, 11]
```

```
k = 5
```

```
count = 0
```

```
num = 1
```

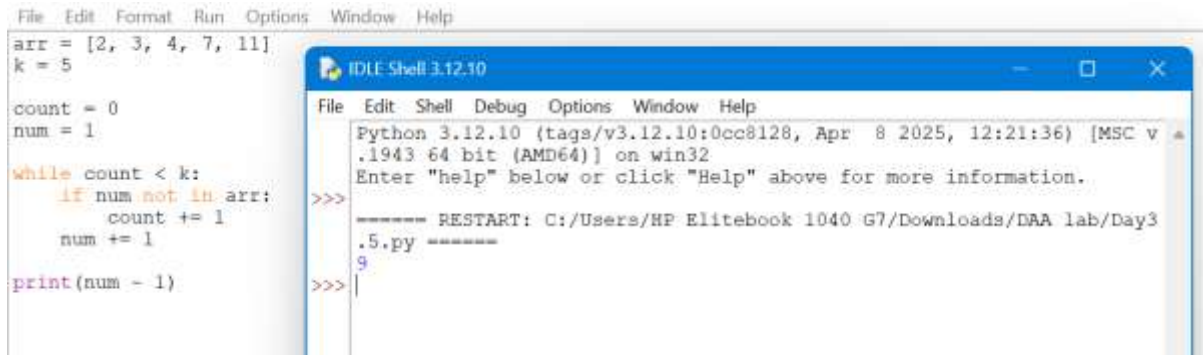
```
while count < k:
```

```
    if num not in arr:
```

```
        count += 1
```

```
num += 1  
print(num - 1)
```

Output:

A screenshot of the Python IDLE Shell interface. The left pane shows a Python script with the following code:

```
arr = [2, 3, 4, 7, 11]  
k = 5  
  
count = 0  
num = 1  
  
while count < k:  
    if num not in arr:  
        count += 1  
        num += 1  
  
print(num - 1)
```

The right pane shows the IDLE Shell output, which includes the Python version (3.12.10), system architecture (64 bit AMD64), and the execution of the script. The output shows the program restarted and then printed the value 9.

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v  
.1943 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
  
===== RESTART: C:/Users/HP Elitebook 1040 G7/Downloads/DAA lab/Day3  
.5.py =====  
9  
>>>
```

6. A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer

array nums, find a peak element, and return its index. If the array contains multiple peaks, return

the index to any of the peaks. You may imagine that $\text{nums}[-1] = \text{nums}[n] = -\infty$. In other words,

an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: $\text{nums} = [1,2,3,1]$

Output: 2

Explanation: 3 is a peak element and your function should return the index number 2.

Example 2:

Input: $\text{nums} = [1,2,1,3,5,6,4]$

Output: 5

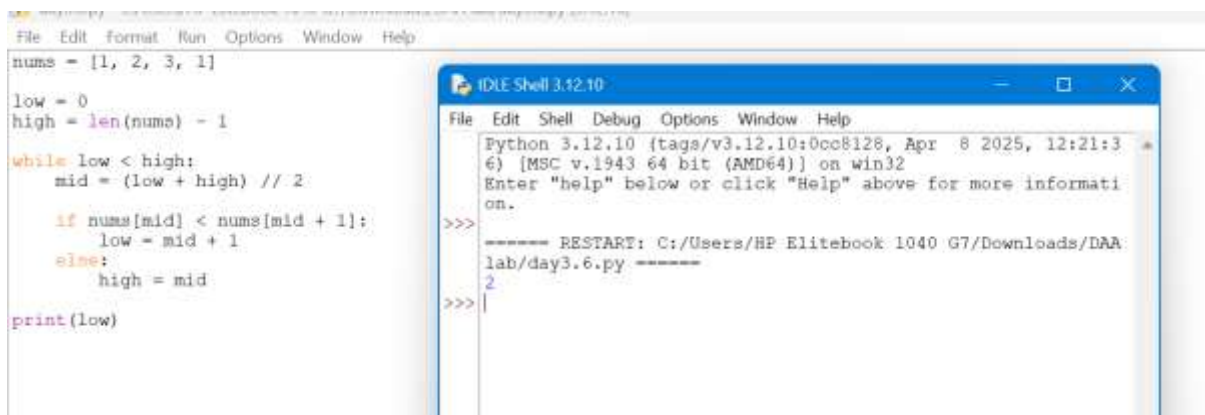
Explanation: Your function can return either index number 1 where the peak element is 2,

or index number 5 where the peak element is 6.

Code:

```
nums = [1, 2, 3, 1]
low = 0
high = len(nums) - 1
while low < high:
    mid = (low + high) // 2
    if nums[mid] < nums[mid + 1]:
        low = mid + 1
    else:
        high = mid
print(low)
```

Output:



7. Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

Example 1:

Input: haystack = "sadbutsad", needle = "sad"

Output: 0

Explanation: "sad" occurs at index 0 and 6.

The first occurrence is at index 0, so we return 0.

Example 2:

Input: haystack = "leetcode", needle = "leeto"

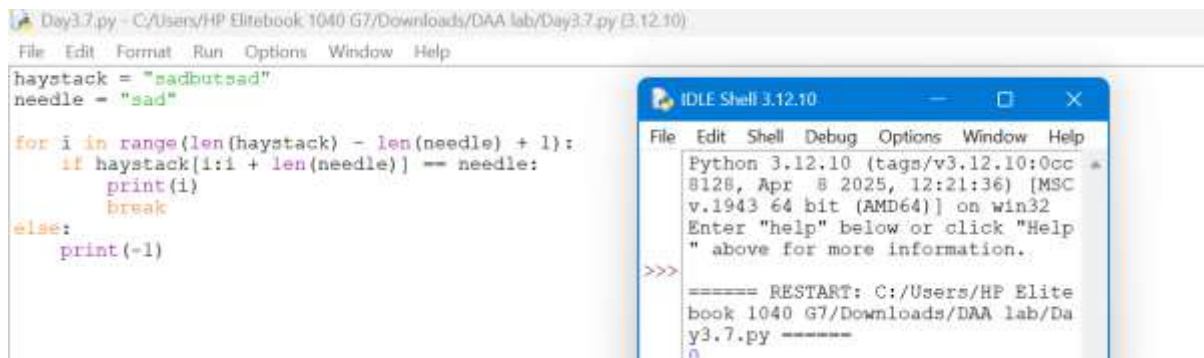
Output: -1

Explanation: "leeto" did not occur in "leetcode", so we return -1.

Code:

```
haystack = "sadbutsad"
needle = "sad"
for i in range(len(haystack) - len(needle) + 1):
    if haystack[i:i + len(needle)] == needle:
        print(i)
        break
else:
    print(-1)
```

Output:



8. Given an array of string words, return all strings in words that is a substring of another word. You

can return the answer in any order. A substring is a contiguous sequence of characters within a

string

Example 1:

Input: words = ["mass","as","hero","superhero"]

Output: ["as","hero"]

Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".

["hero","as"] is also a valid answer.

Example 2:

Input: words = ["leetcode","et","code"]

Output: ["et","code"]

Explanation: "et", "code" are substring of "leetcode".

Example 3:

Input: words = ["blue","green","bu"]

Output: []

Explanation: No string of words is substring of another string.

Code:

```
words = ["mass", "as", "hero", "superhero"]
```

```
result = []
```

```
for i in range(len(words)):
```

```
    for j in range(len(words)):
```

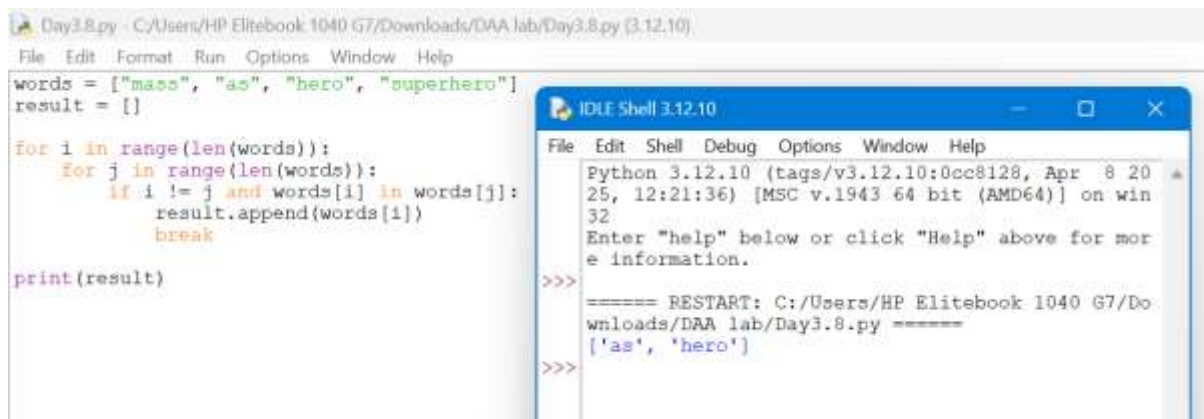
```
        if i != j and words[i] in words[j]:
```

```
            result.append(words[i])
```

```
            break
```

```
print(result)
```

Output:

The image shows a screenshot of a Python IDE with two windows. The left window, titled 'Day3.8.py', contains the following Python code:

```
words = ["mass", "as", "hero", "superhero"]
result = []

for i in range(len(words)):
    for j in range(len(words)):
        if i != j and words[i] in words[j]:
            result.append(words[i])
            break

print(result)
```

The right window, titled 'IDLE Shell 3.12.10', shows the output of the code. It displays the Python version and system information, followed by a restart message and the output list:

```
>>>
===== RESTART: C:/Users/HP Elitebook 1040 G7/Do
wnloads/DAA lab/Day3.8.py =====
['as', 'hero']
>>>
```